



Retrieval-Augmented Generation (RAG)

What it is, why it works, and how to build it

01

Goal

Understand how LLMs can answer using your own documents

02

Focus

Concepts first, implementation second

03

Level

Beginner-friendly with practical engineering details

By the End of This Class

You will walk away with a solid foundation in RAG — from theory to code.

1

Define RAG

Explain what RAG means in simple terms

2

Memory vs. Context

Describe the difference between model memory and retrieved context

3

Core Mechanics

Understand embeddings, vector search, and prompt grounding

4

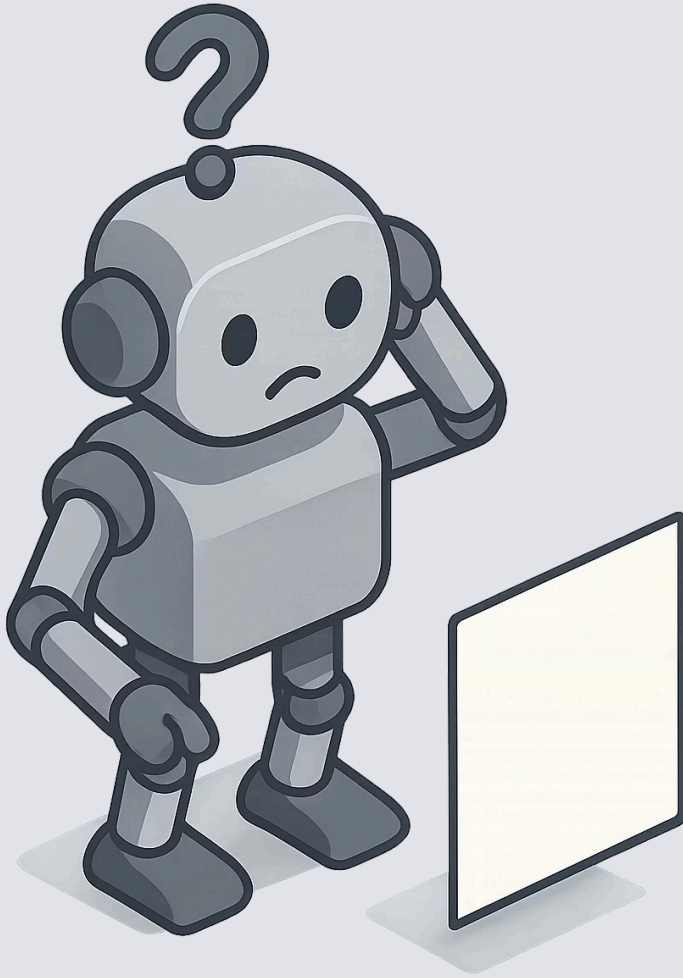
Build a Pipeline

Construct a basic RAG pipeline mentally and in code

5

Failure Modes

Recognize common pitfalls and practical tradeoffs



Why Do We Need RAG?

Outdated Knowledge

LLMs have a training cutoff — they may not know recent events or updates.

No Private Data

They do not automatically know your internal documents or proprietary content.

Hallucination Risk

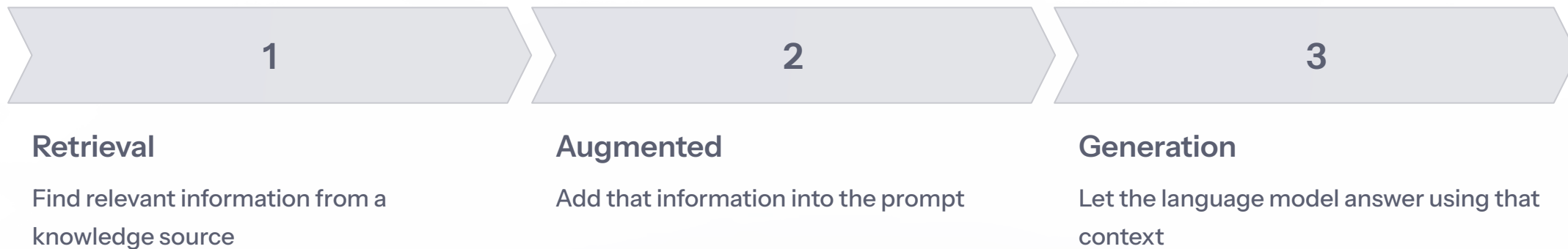
When uncertain, models can generate confident but incorrect answers.



Key idea: RAG gives the model fresh, relevant context at question time — without retraining.

What Is RAG?

RAG stands for **Retrieval-Augmented Generation** — a method that helps an LLM answer questions using retrieved documents instead of relying only on training knowledge.



The RAG Pipeline

Indexing

Chunking

Embeddings

Vector DB

Offline / Indexing Phase

Prepare and store documents in advance — this is batch work done before any user query.

Online / Query Phase

At runtime, retrieve relevant chunks and pass them to the LLM to generate a grounded answer.

Without RAG vs. With RAG

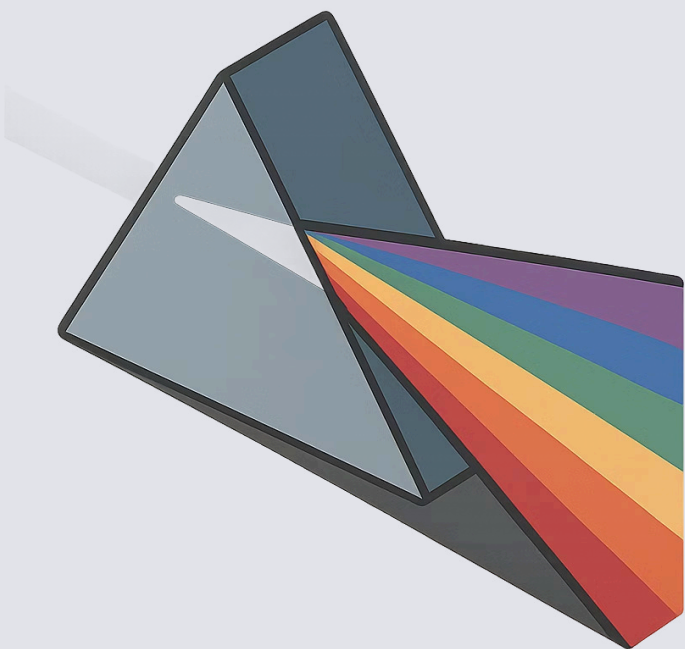
Without RAG

- Model answers from training data only
- May sound confident even when wrong
- Cannot reliably use your latest internal documents

With RAG

- Model receives external context at runtime
- Answers are more grounded and traceable
- Knowledge can be updated without retraining

 Which approach is better for company policy docs or course notes?



What Is an Embedding?

An **embedding** is a vector — a list of numbers that represents the *meaning* of a piece of text. Similar meanings produce vectors that are close together in space, enabling semantic comparison rather than exact word matching.

"How does bread rise?" and "What makes dough expand?" may be very close in vector space — even though they share no words.

Why embeddings matter: They power semantic retrieval — the heart of RAG.

How Retrieval Finds Relevant Text

The Process

- Each document chunk becomes a vector
- The user question also becomes a vector
- The system compares the question vector to stored vectors
- The nearest vectors are treated as the most relevant chunks

Common Similarity Measures

Cosine Similarity

Angle between vectors

Dot Product

Magnitude + direction

Euclidean Distance

Straight-line distance

Why Chunking Matters

Large documents are split into smaller chunks before embedding. Retrieval works better when chunks are focused — smaller chunks reduce irrelevant context, while chunk overlap can preserve continuity across boundaries.

Too Large

Noisy retrieval — the model receives too much irrelevant content alongside the useful parts.

Too Small

Missing context — individual chunks may lack enough information to be useful on their own.

Just Right

Typically **200–800 tokens**, depending on content type and use case.

What Is Stored in a Vector Database?

For each chunk, a vector database stores the following fields:

Core Fields

- **id** — unique identifier
- **text** — original chunk content
- **embedding** — the numeric vector
- **metadata** — structured attributes

Useful Metadata Examples

- title, source, category, date, author, section

Why Metadata Matters

- Filtering, citations, debugging, auditability

A Minimal RAG Architecture

Indexing Phase

1. Load documents
2. Split into chunks
3. Embed chunks
4. Store vectors + metadata

Query Phase

1. Embed the question
2. Retrieve top-k chunks
3. Build the prompt
4. Generate the answer

 **Engineering insight:** Indexing is batch work done in advance. Retrieval and generation happen live at query time.

Code: Create an Embedding

```
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
    model="text-embedding-ada-002",
    input="What is Retrieval-Augmented Generation?"
)

vector = response.data[0].embedding
print(len(vector))
```

→ The output is **numeric**, not natural language

→ Vector length depends on the **embedding model** chosen

→ Embeddings are used for **search**, not for final answering

Code: Store Chunks in a Vector DB

```
collection.add(  
  ids=["doc_001", "doc_002"],  
  documents=[  
    "Fermentation preserves food and develops flavor.",  
    "Quantum computers use qubits and superposition."  
  ],  
  embeddings=[vec1, vec2],  
  metadatas=[  
    {"topic": "cooking", "title": "Fermentation"},  
    {"topic": "technology", "title": "Quantum Computing"}  
  ],  
)
```

Original Text

The raw chunk is stored for display and citation

Embedding

The numeric vector enables similarity search

Metadata

Structured fields support filtering and auditability

Code: Retrieve Relevant Chunks

```
results = collection.query(  
    query_embeddings=[question_vector],  
    n_results=3,  
    include=["documents", "metadatas", "distances"],  
)
```

What Comes Back

- Top matching chunks
- Metadata for each chunk
- Similarity or distance indicators

Important Reminder

Retrieval is **not** the final answer. It selects evidence for the model to reason over. The quality of retrieval directly determines the quality of the answer.

Code: Ground the Model with Context

```
context = "\n\n".join(retrieved_chunks)

system_prompt = f"""
You are a helpful assistant.
Answer using only the context below.
If the answer is not in the context, say you do not know.

Context:
{context}
"""
```

- ✅ **Key lesson:** Prompt design is part of the RAG system. Clear instructions that restrict the model to retrieved context significantly reduce hallucinations.

Full Minimal RAG Example

```
question_vector = embed(user_question)
chunks          = retrieve(question_vector, k=3)
prompt          = build_prompt(user_question, chunks)
answer          = ask_llm(prompt)
```

This looks simple — but production systems require more:



Better Chunking



Metadata Filters



Reranking



Evaluation



Monitoring

Why Teams Use RAG in Practice



Changing Knowledge

Works with frequently updated content — no retraining required for every document update.



Private Data

Can use confidential internal documents at runtime without exposing them during training.



Explainability

Improves traceability — answers can be linked back to source documents.



Cost Effective

Cheaper and faster to update than training or fine-tuning a model from scratch.



RAG Is Powerful, Not Magic

Common failure modes to watch for:

Bad Chunking

Chunks that are too large, too small, or poorly split produce noisy or incomplete retrieval.

Poor Retrieval

Wrong or incomplete documents returned — the model then reasons over bad evidence.

Prompt Leakage

Even with retrieved context, a poorly designed prompt can still allow hallucination.

Weak Evidence

Answer sounds grounded but the retrieved chunks don't actually support the claim.

⊗ **Rule of thumb:** If retrieval fails, generation usually fails after it. Bad retrieval in means bad answers out.

Practical Engineering Choices

Core Design Questions

- Which embedding model should we use?
- How large should chunks be? Use overlap?
- How many chunks should we retrieve?
- Should we rerank retrieved chunks?
- Do we cite sources in the final answer?
- When should we answer "I do not know"?

Advanced Topics

- **Hybrid search:** keyword + vector combined
- **Reranking models:** re-score retrieved chunks
- **Document freshness:** re-indexing strategies
- **Evaluation datasets:** measure retrieval quality
- **Access control:** permissions for private documents

Final Takeaway

The model becomes more useful when it can look up the right information at the right time.

Retrieval

External knowledge fetched at query time

Embeddings

Semantic search over stored document chunks

Grounded Generation

LLM answers using retrieved context, not just training memory

❓ When would you choose RAG over fine-tuning? What happens if the database has wrong information? How would you design chunking for lecture notes?

Classroom Demo Flow

Use this repository as a live demo — run each script in sequence to build intuition step by step.

01

01_what_is_embedding.py

Show vectors and similarity scores

03

03_inspect_vectordb.py

Show what is actually stored

05

05_query_and_retrieve.py

Demonstrate nearest-neighbor search

02

02_store_in_vectordb.py

Build the ChromaDB collection

04

04_visualize_embeddings.py

Make semantic clusters visible

06

06_full_rag_pipeline.py

Compare answers with and without retrieval

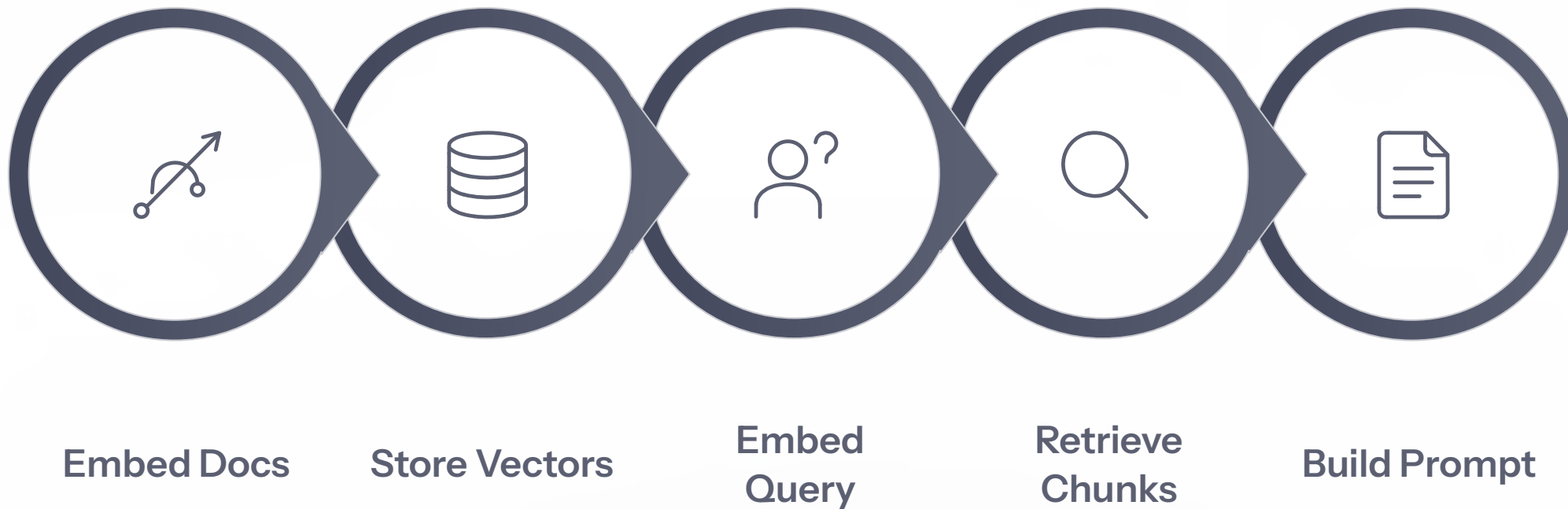
Generic RAG Pseudocode

```
def index_documents(documents):  
    chunks = split_documents(documents)  
    for chunk in chunks:  
        vector = embed(chunk.text)  
        vectordb.add(  
            id=chunk.id,  
            text=chunk.text,  
            embedding=vector,  
            metadata=chunk.metadata,  
        )  
  
def answer_question(question):  
    question_vector = embed(question)  
    retrieved = vectordb.search(question_vector, top_k=3)  
    prompt = build_grounded_prompt(question, retrieved)  
    return llm.generate(prompt)
```

Approach Comparison

Approach	Best For	Weakness
Plain LLM	General knowledge tasks	Not grounded in your data
Fine-tuning	Behavior and style specialization	Not ideal for rapidly changing facts
RAG	External and changing knowledge	Depends heavily on retrieval quality
Fine-tuning + RAG	Specialized behavior plus fresh data	More system complexity

RAG in One Minute



✅ If the system retrieves the right chunks, the model has a much better chance of producing a useful, grounded answer.